



MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Dazy Arya

Subject & Subject Code: CF&P, IFY2-08

Semester: II

Session: 2021-22 (Even Sem.)

Branch: AI&DS, ME, CE, EE

Batch: C,D

UNIT: IV

Date of Submission: 28/03/2023

Lecture No.	Topics Covered	Total Page
1	Arrays, pointers & Structure in C: Arrays-Basic Concepts, one dimensional arrays,	6
2	two dimensional arrays, multidimensional arrays in C, C programming example related to Arrays,.	4
3	Pointer"pointer Arithmetic,	6
4	Programming using Arrays and Pointers	4
5	Size of Operator, an array of pointers,	3
6	, Memory allocation function,	13
7	, command-line argument, pointers to void, Enumeration	5
8	structure and union in C	8

References:

1. Brian W. Kernighan and Dennis Ritchie, C Programming Language, Prentice Hall of India.
2. Yashwant Kanetkar, Let US C, BPB Publication.

Faculty

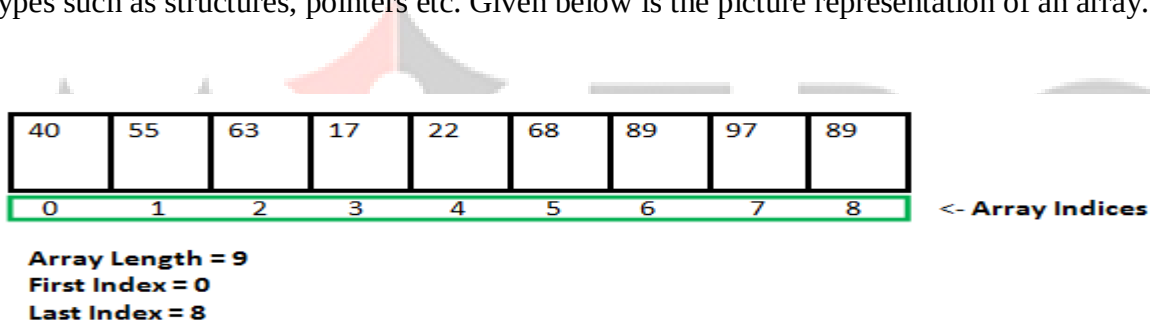
HOD

Dean Academics

LECTURE -01 (UNIT-04)**Syllabus :-Arrays and Pointer****Array**

1. It is a group of variables of similar data types referred to by a single element.
2. Its elements are stored in a contiguous memory location.
3. The size of the array should be mentioned while declaring it.
4. Array elements are always counted from zero (0) onward.
5. Array elements can be accessed using the position of the element in the array.
6. The array can have one or more dimensions.

An array in C/C++ or be it in any programming language is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array. They can be used to store the collection of primitive data types such as int, float, double, char, etc of any particular type. To add to it, an array in C/C++ can store derived data types such as structures, pointers etc. Given below is the picture representation of an array.

**Why do we need arrays?**

We can use normal variables (v1, v2, v3, ..) when we have a small number of objects, but if we want to store a large number of instances, it becomes difficult to manage them with normal variables. The idea of an array is to represent many instances in one variable.

**Advantages:-**

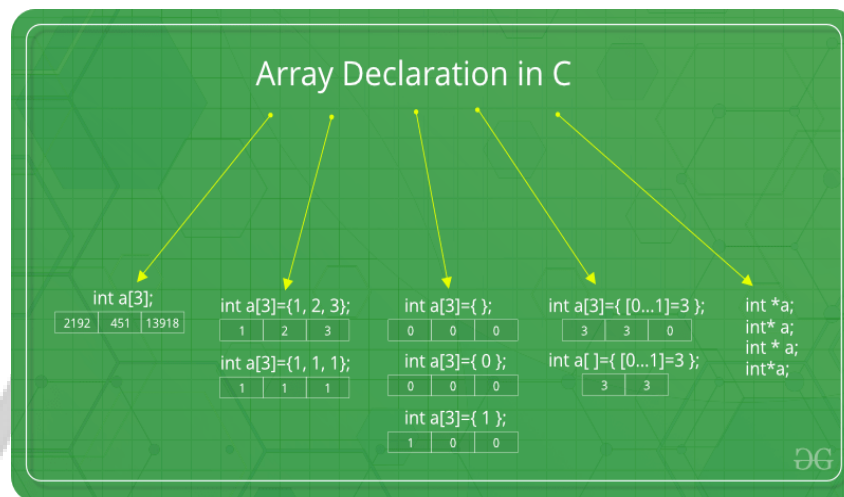
- Code Optimization: we can retrieve or sort the data efficiently.

- Random access: We can get any data located at an index position.

Disadvantages:-

- Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime.

Array declaration in C/C++:



Array declaration by specifying the size

```
#include <iostream>
using namespace std;
```

```
int main()
{
    // array declaration by specifying size
    int arr1[10];

    // With recent C/C++ versions, we can also
    // declare an array of user specified size
    int n = 10;
    int arr2[n];

    return 0;
}
```

// This code is contributed by sarajadhav12052009

Array declaration by initializing elements

```
// Array declaration by initializing elements
#include <iostream>
using namespace std;
int main()
{
    int arr[] = { 10, 20, 30, 40};
    return 0;
// Compiler creates an array of size 4.
// above is same as "int arr[4] = {10, 20, 30, 40}"
}
```

Array declaration by specifying the size and initializing elements

```
#include <iostream>
using namespace std;

int main()
{
    // Array declaration by specifying size and initializing
    // elements
    int arr[6] = { 10, 20, 30, 40 };

    // Compiler creates an array of size 6, initializes first
    // 4 elements as specified by user and rest two elements as
    // 0. above is same as "int arr[] = {10, 20, 30, 40, 0, 0}"

    return 0;
}

// This code is contributed by sarajadhav12052009
```

Advantages of an Array in C/C++:

1. Random access of elements using the array index.
2. Use of fewer lines of code as it creates a single array of multiple elements.
3. Easy access to all the elements.
4. Traversal through the array becomes easy using a single loop.
5. Sorting becomes easy as it can be accomplished by writing fewer lines of code.

Disadvantages of an Array in C/C++:

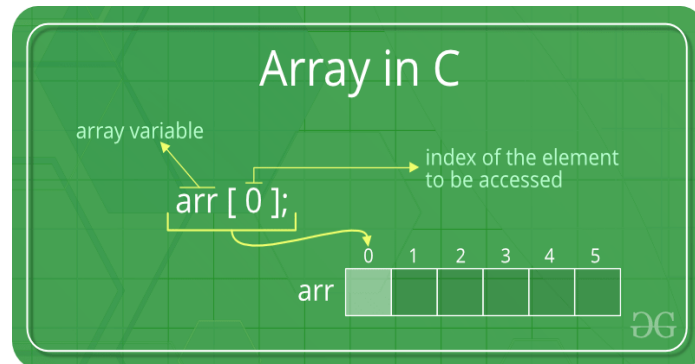
1. Allows a fixed number of elements to be entered which is decided at the time of declaration. Unlike a linked list, an array in C is not dynamic.
2. Insertion and deletion of elements can be costly since the elements are needed to be managed in accordance with the new memory allocation.

Facts about Array in C/C++:

- **Accessing Array Elements:**

Array elements are accessed by using an integer index. Array index starts with 0 and goes till the size of the array minus 1.

- The name of the array is also a pointer to the first element of the array.
- The name of the array is also a pointer to the first element of the array.

**Example:**

```
#include <stdio.h>

int main()
{
    int arr[5];
    arr[0] = 5;
    arr[2] = -10;
    arr[3 / 2] = 2; // this is same as arr[1] = 2
    arr[3] = arr[0];

    printf("%d %d %d %d", arr[0],
           arr[1], arr[2], arr[3]);

    return 0;
}
```

Output

5 2 -10 5

No Index Out of bound Checking:

There is no index out of bounds checking in C/C++, for example, the following program compiles fine but may produce unexpected output when run.

```
// This C program compiles fine
// as index out of bound
// is not checked in C.
```

```
#include <stdio.h>
```

```
int main()
{
    int arr[2];

    printf("%d ", arr[3]);
    printf("%d ", arr[-2]);

    return 0;
}
```

Output

211343841 4195777

In C, it is not a compiler error to initialise an array with more elements than the specified size. For example, the below program compiles fine and shows just a Warning.

```
#include <stdio.h>
int main()
{

    // Array declaration by initializing it
    // with more elements than specified size.
    int arr[2] = { 10, 20, 30, 40, 50 };

    return 0;
}
```

Warnings:

```
prog.c: In function 'main':
prog.c:7:25: warning: excess elements in array initializer
  int arr[2] = { 10, 20, 30, 40, 50 };
                  ^
prog.c:7:25: note: (near initialization for 'arr')
prog.c:7:29: warning: excess elements in array initializer
  int arr[2] = { 10, 20, 30, 40, 50 };
                  ^
prog.c:7:29: note: (near initialization for 'arr')
prog.c:7:33: warning: excess elements in array initializer
  int arr[2] = { 10, 20, 30, 40, 50 };
                  ^
prog.c:7:33: note: (near initialization for 'arr')
```

- **Note:** The program won't compile in C++. If we save the above program as a .cpp, the program generates compiler error "error: too many initializers for 'int [2]'".

The elements are stored at contiguous memory locations**Example:**

```
// C program to demonstrate that
```

```
// array elements are stored
// contiguous locations

#include <stdio.h>
int main()
{
    // an array of 10 integers.
    // If arr[0] is stored at
    // address x, then arr[1] is
    // stored at x + sizeof(int)
    // arr[2] is stored at x +
    // sizeof(int) + sizeof(int)
    // and so on.
    int arr[5], i;

    printf("Size of integer in this compiler is %lu\n",
        sizeof(int));

    for (i = 0; i < 5; i++)
        // The use of '&' before a variable name, yields
        // address of variable.
        printf("Address arr[%d] is %p\n", i, &arr[i]);

    return 0;
}
```

Output

Size of integer in this compiler is 4
Address arr[0] is 0x7fff7a02db20
Address arr[1] is 0x7fff7a02db24
Address arr[2] is 0x7fff7a02db28
Address arr[3] is 0x7fff7a02db2c
Address arr[4] is 0x7fff7a02db30

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -02 (UNIT-04)**Two-Dimensional Array**

Two – dimensional array is the simplest form of a multidimensional array. We can see a two – dimensional array as an array of one-dimensional array for easier understanding.

The basic form of declaring a two-dimensional array of size x, y:

Syntax:

```
data_type array_name[x][y];
```

Here, **data_type** is the type of data to be stored.

We can declare a two-dimensional integer array say 'x' of size 10,20 as:

```
int x[10][20];
```

Elements in two-dimensional arrays are commonly referred to by $x[i][j]$ where i is the row number and ' j ' is the column number.

A two – dimensional array can be seen as a table with 'x' rows and 'y' columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1). A two – dimensional array 'x' with 3 rows and 3 columns is shown below:

	Column 0	Column 1	Column 2
Row 0	$x[0][0]$	$x[0][1]$	$x[0][2]$
Row 1	$x[1][0]$	$x[1][1]$	$x[1][2]$
Row 2	$x[2][0]$	$x[2][1]$	$x[2][2]$

Initializing Two – Dimensional Arrays: There are various ways in which a Two-Dimensional array can be initialized.

First Method:


```
int x[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

The above array has 3 rows and 4 columns. The elements in the braces from left to right are stored in the table also from left to right. The elements will be filled in the array in order, the first 4 elements from the left in the first row, the next 4 elements in the second row, and so on.

Second Method:

```
int x[3][4] = {{0,1,2,3}, {4,5,6,7}, {8,9,10,11}};
```

Third Method:

```
int x[3][4];
for(int i = 0; i < 3; i++){
    for(int j = 0; j < 4; j++){
        cin >> x[i][j];
    }
}
```

Fourth Method(Dynamic Allocation):

```
int** x = new int*[3];
for(int i = 0; i < 3; i++){
    x[i] = new int[4];
    for(int j = 0; j < 4; j++){
        cin >> x[i][j];
    }
}
```

This type of initialization makes use of nested braces. Each set of inner braces represents one row. In the above example, there is a total of three rows so there are three sets of inner braces.

Accessing Elements of Two-Dimensional Arrays: Elements in Two-Dimensional arrays are accessed using the row indexes and column indexes.

Example:

```
int x[2][1];
```

The above example represents the element present in the third row and second column.

Note: In arrays, if the size of an array is N. Its index will be from 0 to N-1. Therefore, for row index 2 row number is $2+1 = 3$. To output all the elements of a Two-Dimensional array we can use nested for loops. We will require two 'for' loops. One to traverse the rows and another to traverse columns.

```
/ C++ Program to print the elements of a
// Two-Dimensional array
#include<iostream>
```

```
using namespace std;

int main()
{
    // an array with 3 rows and 2 columns.
    int x[3][2] = {{0,1}, {2,3}, {4,5}};

    // output each array element's value
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            cout << "Element at x[" << i
                << "][" << j << "]: ";
            cout << x[i][j]<<endl;
        }
    }

    return 0;
}
```

Output:

```
Element at x[0][0]: 0
Element at x[0][1]: 1
Element at x[1][0]: 2
Element at x[1][1]: 3
Element at x[2][0]: 4
Element at x[2][1]: 5
```

Multidimensional Arrays in C

A multi-dimensional array can be termed as an array of arrays that stores homogeneous data in tabular form. Data in multidimensional arrays are stored in row-major order.

The **general form of declaring N-dimensional arrays** is:

```
data_type array_name[size1][size2]...[sizeN];
```

- **data_type:** Type of data to be stored in the array.
- **array_name:** Name of the array
- **size1, size2,...,sizeN:** Sizes of the dimension

Examples:

Two dimensional array: `int two_d[10][20];`

Three dimensional array: `int three_d[10][20][30];`

Size of Multidimensional Arrays:

The total number of elements that can be stored in a multidimensional array can be calculated by multiplying the size of all the dimensions.

For example:

- The array **int x[10][20]** can store total $(10*20) = 200$ elements.
- Similarly array **int x[5][10][20]** can store total $(5*10*20) = 1000$ elements.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.



LECTURE -03 (UNIT-04)

What is Pointer in C?

The **Pointer** in C, is a variable that stores address of another variable. A pointer can also be used to refer to another pointer function. A pointer can be incremented/decremented, i.e., to point to the next/ previous memory location. The purpose of pointer is to save memory space and achieve faster execution time.

How to Use Pointers in C

If we declare a variable `v` of type `int`, `v` will actually store a value.

```
int v = 0;
```

`v` is equal to zero now.

However, each variable, apart from value, also has its address (or, simply put, where it is located in the memory). The address can be retrieved by putting an ampersand (&) before the variable name.

```
&v
```

If you print the address of a variable on the screen, it will look like a totally random number (moreover, it can be different from run to run).

Let's try this in practice with pointer in C example

Pointer Variable

```
int *y = &v;
```

VARIABLE	POINTER
A value stored in a named storage/memory address	A variable that points to the storage/memory address of another variable

Declaring a Pointer

Like variables, pointers in C programming have to be declared before they can be used in your program. Pointers can be named anything you want as long as they obey C's naming rules. A pointer declaration has the following form.

```
data_type * pointer_variable_name;
```

Here,

- **data_type** is the pointer's base type of C's variable types and indicates the type of the variable that the pointer points to.
- The asterisk (*: the same asterisk used for multiplication) which is indirection operator, declares a pointer.

Initialize a pointer

After declaring a pointer, we initialize it like standard variables with a variable address. If pointers in C programming are not uninitialized and used in the program, the results are unpredictable and potentially disastrous.

To get the address of a variable, we use the ampersand (&)operator, placed before the name of a variable whose address we need. Pointer initialization is done with the following syntax.

Pointer Syntax

```
pointer = &variable;
```

A simple program for pointer illustration is given below:

```
#include <stdio.h>
int main()
{
    int a=10; //variable declaration
    int *p;   //pointer variable declaration
    p=&a;      //store address of variable a in pointer p
    printf("Address stored in a variable p is:%x\n",p); //accessing the address
    printf("Value stored in a variable p is:%d\n",*p); //accessing the value
    return 0;
}
```

Output:

```
Address stored in a variable p is:60ff08
Value stored in a variable p is:10
```

Creating Pointers

You learned from the previous chapter, that we can get the **memory address** of a variable with the reference operator &:

Example

```
int myAge = 43; // an int variable
printf("%d", myAge); // Outputs the value of myAge (43)
printf("%p", &myAge); // Outputs the memory address of myAge (0x7ffe5367e044)
```

In the example above, &myAge is also known as a **pointer**.

A **pointer** is a variable that stores the memory address of another variable as its value.

A **pointer variable points to a data type** (like int) of the same type, and is created with the * operator. The address of the variable you are working with is assigned to the pointer:

Example

```
int myAge = 43; // An int variable
int* ptr = &myAge; // A pointer variable, with the name ptr, that stores the address of myAge

// Output the value of myAge (43)
printf("%d\n", myAge);

// Output the memory address of myAge (0x7ffe5367e044)
printf("%p\n", &myAge);

// Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);
```

Example explained

Create a pointer variable with the name ptr, that **points to** an int variable (myAge). Note that the type of the pointer has to match the type of the variable you're working with.

Use the & operator to store the memory address of the myAge variable, and assign it to the pointer.

Now, ptr holds the value of myAge's memory address.

Dereference

In the example above, we used the pointer variable to get the memory address of a variable (used together with the **& reference** operator).

However, you can also get the value of the variable the pointer points to, by using the ***** operator (the **dereference** operator):

Example

```
int myAge = 43;    // Variable declaration
int* ptr = &myAge; // Pointer declaration

// Reference: Output the memory address of myAge with the pointer (0x7ffe5367e044)
printf("%p\n", ptr);

// Dereference: Output the value of myAge with the pointer (43)
printf("%d\n", *ptr);
```

Note that the ***** sign can be confusing here, as it does two different things in our code:

- When used in declaration (`int* ptr`), it creates a **pointer variable**.
- When not used in declaration, it act as a **dereference operator**.

Why Should I Learn About Pointers?

Pointers are one of the things that make C stand out from other programming languages, like Python and Java.

This chapter was just a short introduction to Pointers. They are important in C, because they give you the ability to manipulate the data in the computer's memory - this can reduce the code and improve the performance. **However**, pointers must be handled with care, since it is possible to damage data stored in other memory addresses.

Pointers variables are also known as **address data types** because they are used to store the address of another variable. The address is the memory location that is assigned to the variable. It doesn't store any value.

Pointer Arithmetic on Arrays:

Pointers contain addresses. Adding two addresses makes no sense because there is no idea what it would point to. Subtracting two addresses lets you compute the offset between the two addresses. An array name acts like a pointer constant. The value of this pointer constant is the

address of the first element. **For Example:** if an array named arr then arr and &arr[0] can be used to reference array as a pointer.

Below is the program to illustrate the Pointer Arithmetic on arrays:

Program 1:

```
// C program to illustrate the array
// traversal using pointers
#include <stdio.h>
```

```
// Driver Code
int main()
{
```

```
    int N = 5;
```

```
    // An array
    int arr[] = { 1, 2, 3, 4, 5 };
```

```
    // Declare pointer variable
    int* ptr;
```

```
    // Point the pointer to first
    // element in array arr[]
    ptr = arr;
```

```
    // Traverse array using ptr
    for (int i = 0; i < N; i++) {
```

```
        // Print element at which
        // ptr points
        printf("%d ", ptr[0]);
        ptr++;
    }
```

```
}
```

Output

1 2 3 4 5

Program 2:

```
// C program to illustrate the array
// traversal using pointers in 2D array
#include <stdio.h>
```

```
// Function to traverse 2D array
```



```
// using pointers
void traverseArr(int* arr,
                int N, int M)
{
    int i, j;

    // Traverse rows of 2D matrix
    for (i = 0; i < N; i++) {

        // Traverse columns of 2D matrix
        for (j = 0; j < M; j++) {

            // Print the element
            printf("%d ", *((arr + i * M) + j));
        }
        printf("\n");
    }
}

// Driver Code
int main()
{
    int N = 3, M = 2;

    // A 2D array
    int arr[][2] = { { 1, 2 },
                    { 3, 4 },
                    { 5, 6 } };

    // Function Call
    traverseArr((int*)arr, N, M);
    return 0;
}
```

Output

```
1 2
3 4
5 6
```

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -04 (UNIT-04)

Array and Pointers in C Language hold a very strong relationship. Generally, pointers are the variables which contain the addresses of some other variables and with arrays a pointer stores the starting address of the array. Array name itself acts as a pointer to the first element of the array and also if a pointer variable stores the base address of an array then we can manipulate all the array elements using the pointer variable only. Pointers can be associated with the multidimensional arrays (2-D and 3-D arrays) as well. Also, We can create an array of pointers to store multiple addresses of different variables.

Arrays can be single or multidimensional and are stored in contiguous memory blocks in our system, so it is easy for pointers to get associated with the arrays.

An array name is generally treated as a pointer to the first element of the array and if we store the base address of the array in another pointer variable then we can easily manipulate the array using pointer arithmetic in a C Program.

we can declare an integer array using the below statement :

```
int arr[5];
```

The above statement will allocate 555 integer blocks and will occupy a memory of 20 Bytes in the system ($5 * 4 = 20$, 5 is size of the array and 444 bytes is the space occupied by an integer block, total = 20).

Below is the representation of how the array is stored in the system's memory. Let the base address allocated by the system to the array is **300**.

Relationship between Pointers and Arrays in C

In respect to the pointers concept, let us see some important points related to arrays in general :

- 'arr' serves two purposes here, first it is the name of the array and second arr itself represents the base address of the array i.e. 300300300 in the above case, if we print the value in arr then it will print the address of the first element in the array.
- As the array name arr itself represents the base address of the array, then by default arr acts as a pointer to the first element of the array.
- arr is the same as &arr and &arr[0] in C Language.
- If we use dereferencing operator (*) on any of the above representations of array address we will get the value of the very first element of the array.

Let us look at the program below to see arr, &arr and &arr[0] means the same.

C Program :

C Program :

```
#include <stdio.h>

int main()
{
    // array declaration and initialization
    int arr[5] = {3, 5, 7, 9, 11};

    // printing the addresses and values represented by arr, &arr and &arr[0]
    printf("arr : %u, Value : %d\n", arr, *arr);

    printf("&arr : %u, Value : %d\n", &arr, *(arr));

    printf("&arr[0] : %u, Value : %d\n", &arr[0], *(&arr[0]));

    return 0;
}
```

Output:

```
[Success] Your code was executed successfully
arr : 63744176, Value : 3
&arr : 63744176, Value : 3
&arr[0] : 63744176, Value : 3
```

Note : Output address will be different at every run.

You can run and check your code here. (IDE by InterviewBit)

We can see that arr, &arr and &arr[0] are printing the same addresses and values in the output window. So, it is clear from the above program and output that arr, &arr and &arr[0] represent the same address in the system's memory.

Syntax Representing Array in Terms of Pointers in C

In a C Program, we denote array elements as arr[i], where i is the index value. Below is a similar syntax in terms of **pointers** of how we can represent the array elements using the dereferencing operator (*) on the array name i.e. using the pointers property of the array.

```
*(arr + i)
```

- * is a dereferencing operator used to extract the value from the address (arr + i).
- *(arr + i) is the same as arr[i] in a C Program.
- arr represents the array name and i represents the index value.

- **Example**

- Let us look at a program to print the values and address of the array elements using the above **syntax**.

- **C Program :**

```
#include <stdio.h>

int main()
{
    // array declaration and initialization
    int arr[5] = {2, 4, 6, 8, 10}, i;

    for(i = 0; i < 5; i++)
    {
        // printing the elements address and value at
        // arr[i] using *(arr + i) syntax
        printf("[index %d] Address : %u, Value : %d\n", i, (arr + i), *(arr + i));
    }

    return 0;
}
```

- **OUTPUT :**

```
[Success] Your code was executed successfully

[index 0] Address : 2364420656, Value : 2
[index 1] Address : 2364420660, Value : 4
[index 2] Address : 2364420664, Value : 6
[index 3] Address : 2364420668, Value : 8
[index 4] Address : 2364420672, Value : 10
```

Explanation :

- We have declared and initialized an integer array arr, array representation :
- □ (arr + i) represents the address of the value at index i, so *(arr + i) will give the value at ith index (address(arr + i) = address(arr[i])), it is used to print the addresses of the array elements as the value of i changes from 0-4.
- □ * is a dereferencing operator used for printing the value at the provided address. *(arr + i) will print the values of the array at consecutive addresses as the value of i changes from 0-4

Note : From the above example we can conclude that, &arr[0] is equal to arr and arr[0] is equal to *arr. Similarly,

- &arr[1] is equal to (arr + 1) and arr[1] is equal to *(arr + 1).
- &arr[2] is equal to (arr + 2) and arr[2] is equal to *(arr + 2) and so on.
- ...
- Finally, we can write the above expressions in a fundamental form as :
- &arr[i] is equal to (arr + i) and arr[i] is equal to *(arr + i).

Note : When the array name arr is an operand of sizeof() operator or the & (address-of) unary operator i.e. sizeof(arr) and &arr respectively, then the array name arr refers to the whole array object, thus sizeof(arr) gives us the size of the entire array in bytes and &arr covers the whole

array because as we know the array name `arr` generally means the base address of the array, so `arr` and `&arr` are equivalent but `arr + 1` and `&arr + 1` will not be equal if the array size is more than 1, `arr + 1` gives the address of the next element in the array, while `&arr + 1` gives the address of the element that is next to the last element of the array (`&arr` covers the whole array).

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.



LECTURE -05 (UNIT-04)

Sizeof is a much used operator in the C or C++. It is a compile time unary operator which can be used to compute the size of its operand. The result of sizeof is of unsigned integral type which is usually denoted by size_t. sizeof can be applied to any data-type, including primitive types such as integer and floating-point types, pointer types, or compound datatypes such as Structure, union etc.

Usage

sizeof() operator is used in different way according to the operand type.

1. When operand is a Data Type.

When *sizeof()* is used with the data types such as int, float, char... etc it simply returns the amount of memory is allocated to that data types.

Let's see example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("%lu\n", sizeof(char));
```

```
    printf("%lu\n", sizeof(int));
```

```
    printf("%lu\n", sizeof(float));
```

```
    printf("%lu", sizeof(double));
```

```
    return 0;
```

```
}
```

Output

```
1
```

```
4
```

```
4
```

```
8
```

Note: sizeof() may give different output according to machine, we have run our program on 32 bit gcc compiler.

2. When operand is an expression.

When *sizeof()* is used with the expression, it returns size of the expression. Let see example:

```
#include <stdio.h>
int main()
{
    int a = 0;
    double d = 10.21;
    printf("%lu", sizeof(a + d));
    return 0;
}
```

Output

8

Pointer to Array in C

In a pointer to an array, we just have to store the base address of the array into the pointer variable. We know in the arrays that the base address of an array can be represented in three forms, let us see these syntax of how we can store the base address in a pointer variable:

- `*ptr = &arr;`

-

- `*ptr = arr;`

-

- `*ptr = &arr[0];`

-

- In all the above cases, ptr will store the base address of the array. Now, Let's see an example where we are printing array elements using a pointer to array. We will add consecutive integer values to the pointer ptr using a for loop and with the help of addition arithmetic we are going to print the array elements.

- **C Program :**

```
#include <stdio.h>
int main()
{
    // array declaration and initialization
    int arr[5] = {3, 5, 7, 9, 11}, i;

    // both `arr` and `&arr` return the address of the first element of the array.
    int *ptr = arr;

    // printing the elements of array using addition arithmetic on pointer
    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }

    return 0;
}
```

- **OUTPUT :**

- [Success] Your code was executed successfully
- 3 5 7 9 11

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.



LECTURE -06 (UNIT-04)

Dynamic Memory Allocation in C using malloc(), calloc(), free() and realloc()

- Difficulty Level : Easy
- Last Updated : 09 Dec, 2021

- ☐ Read
- ☐ Discuss

Since C is a structured language, it has some fixed rules for programming. One of them includes changing the size of an array. An array is a collection of items stored at contiguous memory locations.

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

As it can be seen that the length (size) of the array above made is 9. But what if there is a requirement to change this length (size). For Example,

- If there is a situation where only 5 elements are needed to be entered in this array. In this case, the remaining 4 indices are just wasting memory in this array. So there is a requirement to lessen the length (size) of the array from 9 to 5.
- Take another situation. In this, there is an array of 9 elements with all 9 indices filled. But there is a need to enter 3 more elements in this array. In this case, 3 indices more are required. So the length (size) of the array needs to be changed from 9 to 12.

This procedure is referred to as **Dynamic Memory Allocation in C**.

Therefore, C **Dynamic Memory Allocation** can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.

C provides some functions to achieve these tasks. There are 4 library functions provided by C defined under `<stdlib.h>` header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()
4. realloc()

Let's look at each of them in greater detail.

C malloc() method

The “**malloc**” or “**memory allocation**” method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. It doesn't Initialize memory at execution time so that it has initialized each block with the default garbage value initially.

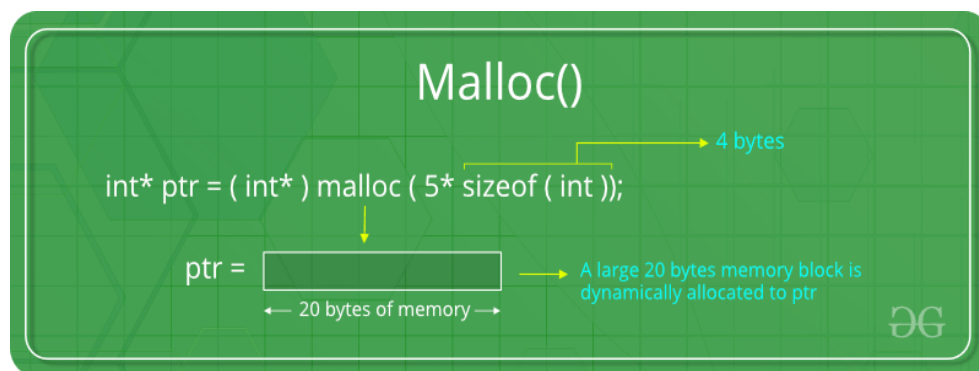
Syntax:

```
ptr = (cast-type*) malloc(byte-size)
```

For Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.



If space is insufficient, allocation fails and returns a NULL pointer.

Example:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    // This pointer will hold the
```

```
    // base address of the block created
```

```
    int* ptr;
```

```
int n, i;

// Get the number of elements for the array
printf("Enter number of elements:");
scanf("%d",&n);

printf("Entered number of elements: %d\n", n);

// Dynamically allocate memory using malloc()
ptr = (int*)malloc(n * sizeof(int));

// Check if the memory has been successfully
// allocated by malloc or not
if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {
    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc.\n");

    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}
```

```
    }  
}  
  
    return 0;  
  
}
```

Output:

Enter number of elements: 5
Memory successfully allocated using malloc.
The elements of the array are: 1, 2, 3, 4, 5,

C calloc() method

1. **“calloc” or “contiguous allocation”** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but has two different points and these are:
2. It initializes each block with a default value ‘0’.
3. It has two parameters or arguments as compare to malloc().

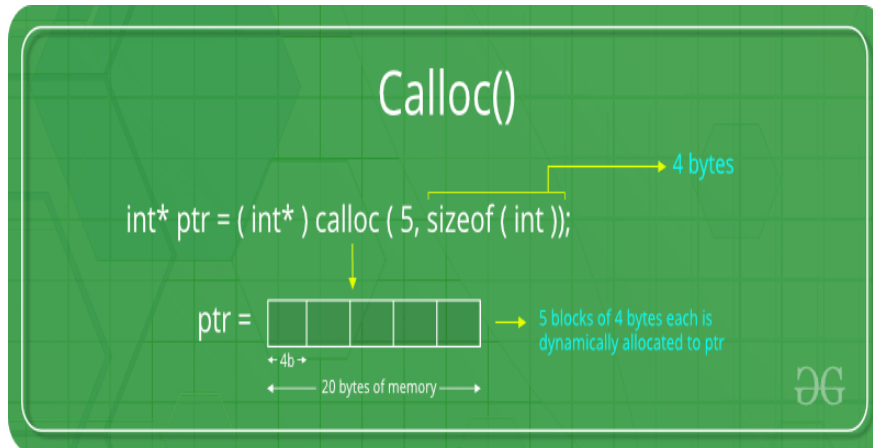
Syntax:

`ptr = (cast-type*)calloc(n, element-size);`
here, n is the no. of elements and element-size is the size of each element.

For Example:

`ptr = (float*) calloc(25, sizeof(float));`

This statement allocates contiguous space in memory for 25 elements each with the size of the float.



If space is insufficient, allocation fails and returns a NULL pointer.

Example:

```
#include <stdio.h>

#include <stdlib.h>

int main()
{
    // This pointer will hold the
    // base address of the block created

    int* ptr;

    int n, i;

    // Get the number of elements for the array
    n = 5;

    printf("Enter number of elements: %d\n", n);

    // Dynamically allocate memory using calloc()
    ptr = (int*)calloc(n, sizeof(int));

    // Check if the memory has been successfully
    // allocated by calloc or not
```

```
if (ptr == NULL) {  
    printf("Memory not allocated.\n");  
    exit(0);  
}  
else {  
    // Memory has been successfully allocated  
    printf("Memory successfully allocated using calloc.\n");  
    // Get the elements of the array  
    for (i = 0; i < n; ++i) {  
        ptr[i] = i + 1;  
    }  
    // Print the elements of the array  
    printf("The elements of the array are: ");  
    for (i = 0; i < n; ++i) {  
        printf("%d, ", ptr[i]);  
    }  
}  
return 0;  
}
```

Output:

Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

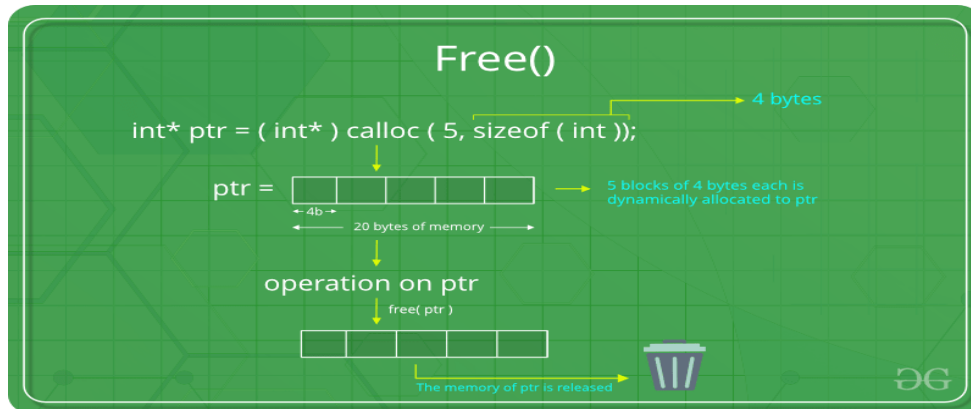
C free() method

“**free**” method in C is used to dynamically **de-allocate** the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own. Hence the free() method is used,

whenever the dynamic memory allocation takes place. It helps to reduce wastage of memory by freeing it.

Syntax:

```
free(ptr);
```



Example:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // This pointer will hold the
    // base address of the block created

    int *ptr, *ptr1;

    int n, i;

    // Get the number of elements for the array

    n = 5;

    printf("Enter number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()

    ptr = (int*)malloc(n * sizeof(int));
```

```
// Dynamically allocate memory using calloc()

ptr1 = (int*)calloc(n, sizeof(int));

// Check if the memory has been successfully
// allocated by malloc or not

if (ptr == NULL || ptr1 == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {
    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc.\n");

    // Free the memory
    free(ptr);
    printf("Malloc Memory successfully freed.\n");

    // Memory has been successfully allocated
    printf("\nMemory successfully allocated using calloc.\n");

    // Free the memory
    free(ptr1);
    printf("Calloc Memory successfully freed.\n");
}

return 0;
}
```

Output:

```
Enter number of elements: 5
Memory successfully allocated using malloc.
Malloc Memory successfully freed.
```


Memory successfully allocated using calloc.
Calloc Memory successfully freed.

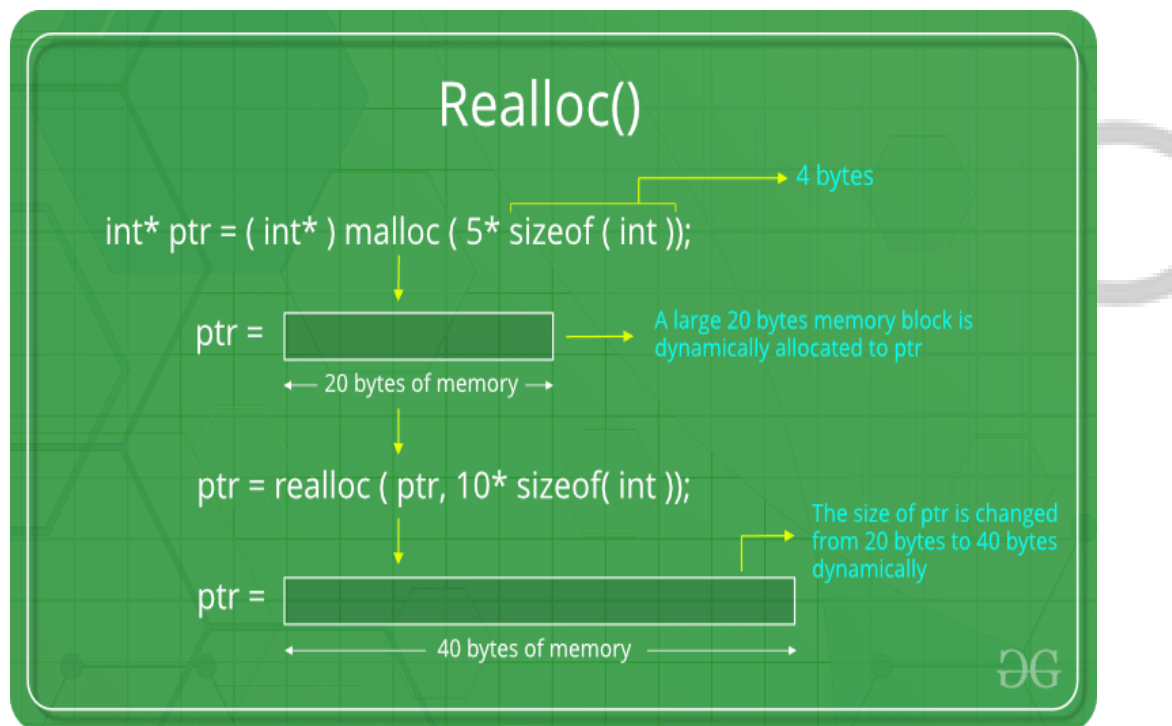
C realloc() method

“**realloc**” or “**re-allocation**” method in C is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**. re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax:

```
ptr = realloc(ptr, newSize);
```

where ptr is reallocated with new size 'newSize'.



If space is insufficient, allocation fails and returns a NULL pointer.

Example:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    // This pointer will hold the
```

```
    // base address of the block created
```

```
    int* ptr;
```

```
    int n, i;
```

```
    // Get the number of elements for the array
```

```
    n = 5;
```

```
    printf("Enter number of elements: %d\n", n);
```

```
    // Dynamically allocate memory using calloc()
```

```
    ptr = (int*)calloc(n, sizeof(int));
```

```
    // Check if the memory has been successfully
```

```
    // allocated by malloc or not
```

```
    if (ptr == NULL) {
```

```
        printf("Memory not allocated.\n");
```

```
        exit(0);
```

```
    }
```

```
    else {
```

```
        // Memory has been successfully allocated
```

```
        printf("Memory successfully allocated using calloc.\n");
```

```
        // Get the elements of the array
```

```
for (i = 0; i < n; ++i) {
    ptr[i] = i + 1;
}

// Print the elements of the array
printf("The elements of the array are: ");

for (i = 0; i < n; ++i) {
    printf("%d, ", ptr[i]);
}

// Get the new size for the array
n = 10;

printf("\n\nEnter the new size of the array: %d\n", n);

// Dynamically re-allocate memory using realloc()
ptr = realloc(ptr, n * sizeof(int));
// Memory has been successfully allocated
printf("Memory successfully re-allocated using realloc.\n");

// Get the new elements of the array
for (i = 5; i < n; ++i) {
    ptr[i] = i + 1;
}

// Print the elements of the array
printf("The elements of the array are: ");

for (i = 0; i < n; ++i) {
    printf("%d, ", ptr[i]);
}
```

```
    free(ptr);  
  
}  
  
    return 0;  
  
}
```

Output:

Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

Enter the new size of the array: 10
Memory successfully re-allocated using realloc.
The elements of the array are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -07 (UNIT-04)

Command Line Argument

So far, we have seen that no arguments were passed in the main function. But the C programming language gives the programmer the provision to add parameters or arguments inside the main function to reduce the length of the code. These arguments are called command line arguments in C.

Command line arguments are nothing but simply arguments that are specified after the name of the program in the system's command line, and these argument values are passed on to your program during program execution.

The basic syntax is:

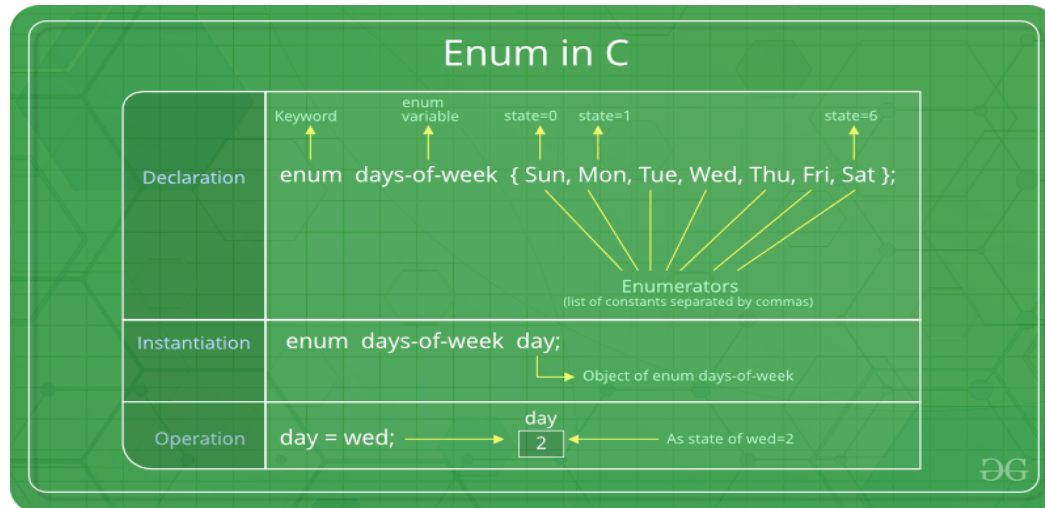
```
int main( int argc, char *argv[] )
{
.
.
// BODY OF THE MAIN FUNCTION
.
.
}
```

Now, let us discuss the components of this code in detail:

- **argc:** It refers to “argument count”. It is the first parameter that we use to store the number of command line arguments. It is important to note that the value of argc should be greater than or equal to 0.
- **argv:** It refers to “argument vector”. It is basically an array of character pointer which we use to list all the command line arguments.
- // The program name is cl.c
- #include<stdio.h>
- int main(int argc, char** argv)
- {
- printf("Welcome to DataFlair tutorials!\n\n");
- int i;
- printf("The number of arguments are: %d\n",argc);
- printf("The arguments are:");
- for (i = 0; i < argc; i++)
- {
- printf("%s\n", argv[i]);
- }
- return 0;
- }

Enumeration (or enum) in C

Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.



```
enum State {Working = 1, Failed = 0};
```

The keyword 'enum' is used to declare new enumeration types in C and C++. Following is an example of enum declaration.

```
// The name of enumeration is "flag" and the constant
// are the values of the flag. By default, the values
// of the constants are as follows:
// constant1 = 0, constant2 = 1, constant3 = 2 and
// so on.
enum flag{constant1, constant2, constant3, ..... };
```

Variables of type enum can also be defined. They can be defined in two ways:

```
// In both of the below cases, "day" is
// defined as the variable of type week.
```

```
enum week{Mon, Tue, Wed};
enum week day;
```

```
// Or
```

```
enum week{Mon, Tue, Wed}day;
```

```
// An example program to demonstrate working
// of enum in C
#include<stdio.h>

enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};

int main()
{
    enum week day;
    day = Wed;
    printf("%d",day);
    return 0;
}
```

Output:

2

In the above example, we declared “day” as the variable and the value of “Wed” is allocated to day, which is 2. So as a result, 2 is printed.
Another example of enumeration is:

```
// Another example program to demonstrate working
// of enum in C
#include<stdio.h>

enum year{Jan, Feb, Mar, Apr, May, Jun, Jul,
          Aug, Sep, Oct, Nov, Dec};

int main()
{
    int i;
    for (i=Jan; i<=Dec; i++)
        printf("%d ", i);

    return 0;
}
```

Output:

0 1 2 3 4 5 6 7 8 9 10 11

In this example, the for loop will run from $i = 0$ to $i = 11$, as initially the value of i is Jan which is 0 and the value of Dec is 11.

Interesting facts about initialization of enum.

1. Two enum names can have same value. For example, in the following C program both 'Failed' and 'Freezed' have same value 0.

VOID POINTER

The void pointer in C is a pointer which is not associated with any data types. It points to some data location in the storage means points to the address of variables. It is also called general purpose pointer. In C, malloc() and calloc() functions return void * or generic pointers.

It has some limitations –

- 1) Pointer arithmetic is not possible with void pointer due to its concrete size.
- 2) It can't be used as dereferenced.

Algorithm

Begin

 Declare a of the integer datatype.

 Initialize a = 7.

 Declare b of the float datatype.

 Initialize b = 7.6.

 Declare a pointer p as void.

 Initialize p pointer to a.

 Print "Integer variable is".

 Print the value of a using pointer p.

 Initialize p pointer to b.

 Print "Float variable is".

 Print the value of b using pointer p

End.

Here is a simple example –

Example Code

Live Demo

```
#include<stdlib.h>
int main() {
    int a = 7;
```



```
float b = 7.6;
void *p;
p = &a;
printf("Integer variable is = %d", *( (int*) p) );
p = &b;
printf("\nFloat variable is = %f", *( (float*) p) );
return 0;
}
```

Output

Integer variable is = 7
Float variable is = 7.600000

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

**LECTURE -08 (UNIT-04)**

C Structures (structs)

Structures (also called structs) are a way to group several related variables into one place. Each variable in the structure is known as a **member** of the structure.

Unlike an array, a structure can contain many different data types (int, float, char, etc.).

Create a Structure

You can create a structure by using the `struct` keyword and declare each of its members inside curly braces:

```
struct MyStructure { // Structure declaration
    int myNum;        // Member (int variable)
    char myLetter;     // Member (char variable)
}; // End the structure with a semicolon
```

To access the structure, you must create a variable of it.

Use the `struct` keyword inside the `main()` method, followed by the name of the structure and then the name of the structure variable:

Create a struct variable with the name "s1":

```
struct myStructure {
    int myNum;
    char myLetter;
};

int main() {
    struct myStructure s1;
    return 0;
}
```

Access Structure Members

To access members of a structure, use the dot syntax (.):

Example

```
// Create a structure called myStructure
struct myStructure {
```

```
int myNum;
char myLetter;
};

int main() {
    // Create a structure variable of myStructure called s1
    struct myStructure s1;

    // Assign values to members of s1
    s1.myNum = 13;
    s1.myLetter = 'B';

    // Print values
    printf("My number: %d\n", s1.myNum);
    printf("My letter: %c\n", s1.myLetter);

    return 0;
}
```

Now you can easily create multiple structure variables with different values, using just one structure:

Example

```
// Create different struct variables
struct myStructure s1;
struct myStructure s2;

// Assign values to different struct variables
s1.myNum = 13;
s1.myLetter = 'B';

s2.myNum = 20;
s2.myLetter = 'C';
```

What About Strings in Structures?

Remember that strings in C are actually an array of characters, and unfortunately, you can't assign a value to an array like this:

Example

```
struct myStructure {
    int myNum;
```

```
char myLetter;
char myString[30]; // String
};

int main() {
    struct myStructure s1;

    // Trying to assign a value to the string
    s1.myString = "Some text";

    // Trying to print the value
    printf("My string: %s", s1.myString);

    return 0;
}
```

An error will occur:

prog.c:12:15: error: assignment to expression with array type

However, there is a solution for this! You can use the `strcpy()` function and assign the value to `s1.myString`, like this:

Example

```
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30]; // String
};

int main() {
    struct myStructure s1;

    // Assign a value to the string using the strcpy function
    strcpy(s1.myString, "Some text");

    // Print the value
    printf("My string: %s", s1.myString);

    return 0;
}
```

Result:

My string: Some text

Simpler Syntax

You can also assign values to members of a structure variable at declaration time, in a single line.

Just insert the values in a comma-separated list inside curly braces {}. Note that you don't have to use the strcpy() function for string values with this technique:

Example

```
// Create a structure
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30];
};

int main() {
    // Create a structure variable and assign values to it
    struct myStructure s1 = {13, 'B', "Some text"};

    // Print values
    printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

    return 0;
}
```

Copy Structures

You can also assign one structure to another.

In the following example, the values of s1 are copied to s2:

Example

```
struct myStructure s1 = {13, 'B', "Some text"};
struct myStructure s2;

s2 = s1;
```

Modify Values

If you want to change/modify a value, you can use the dot syntax (.).

And to modify a string value, the strcpy() function is useful again:

Example

```
struct myStructure {
    int myNum;
    char myLetter;
    char myString[30];
};

int main() {
    // Create a structure variable and assign values to it
    struct myStructure s1 = {13, 'B', "Some text"};

    // Modify values
    s1.myNum = 30;
    s1.myLetter = 'C';
    strcpy(s1.myString, "Something else");

    // Print values
    printf("%d %c %s", s1.myNum, s1.myLetter, s1.myString);

    return 0;
}
```

Modifying values are especially useful when you copy structure values:

Example

```
// Create a structure variable and assign values to it
struct myStructure s1 = {13, 'B', "Some text"};

// Create another structure variable
struct myStructure s2;

// Copy s1 values to s2
s2 = s1;

// Change s2 values
s2.myNum = 30;
```

```
s2.myLetter = 'C';
strcpy(s2.myString, "Something else");

// Print values
printf("%d %c %s\n", s1.myNum, s1.myLetter, s1.myString);
printf("%d %c %s\n", s2.myNum, s2.myLetter, s2.myString);
```

, how are structures useful?

Imagine you have to write a program to store different information about Cars, such as brand, model, and year. What's great about structures is that you can create a single "Car template" and use it for every cars you make. See below for a real life example.

Real Life Example

Use a structure to store different information about Cars:

Example

```
struct Car {
    char brand[50];
    char model[50];
    int year;
};

int main() {
    struct Car car1 = {"BMW", "X5", 1999};
    struct Car car2 = {"Ford", "Mustang", 1969};
    struct Car car3 = {"Toyota", "Corolla", 2011};

    printf("%s %s %d\n", car1.brand, car1.model, car1.year);
    printf("%s %s %d\n", car2.brand, car2.model, car2.year);
    printf("%s %s %d\n", car3.brand, car3.model, car3.year);

    return 0;
}
```

Union in C

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location. The union can also be defined as many members, but only one member can contain a value at a particular point in time.

Union is a user-defined data type, but unlike structures, they share the same memory location.

Let's understand this through an example.

```
1. struct abc
2. {
3.     int a;
4.     char b;
5. }
```

The above code is the user-defined structure that consists of two members, i.e., 'a' of type **int** and 'b' of type **character**. When we check the addresses of 'a' and 'b', we found that their addresses are different. Therefore, we conclude that the members in the structure do not share the same memory location.

When we define the union, then we found that union is defined in the same way as the structure is defined but the difference is that union keyword is used for defining the union data type, whereas the struct keyword is used for defining the structure. The union contains the data members, i.e., 'a' and 'b', when we check the addresses of both the variables then we found that both have the same addresses. It means that the union members share the same memory location.

Let's have a look at the pictorial representation of the memory allocation.

The below figure shows the pictorial representation of the structure. The structure has two members; i.e., one is of integer type, and the another one is of character type. Since 1 block is equal to 1 byte; therefore, 'a' variable will be allocated 4 blocks of memory while 'b' variable will be allocated 1 block of memory.

The below figure shows the pictorial representation of union members. Both the variables are sharing the same memory location and having the same initial address.

In union, members will share the memory location. If we try to make changes in any of the member then it will be reflected to the other member as well. Let's understand this concept through an example.

```
1. union abc
2. {
3.     int a;
4.     char b;
5. }var;
6. int main()
7. {
8.     var.a = 66;
9.     printf("\n a = %d", var.a);
10.    printf("\n b = %d", var.b);
11. }
```


In the above code, union has two members, i.e., 'a' and 'b'. The 'var' is a variable of union abc type. In the **main()** method, we assign the 66 to 'a' variable, so var.a will print 66 on the screen. Since both 'a' and 'b' share the memory location, **var.b** will print '**B**' (ascii code of 66).

Deciding the size of the union

The size of the union is based on the size of the largest member of the union.

Let's understand through an example.

```
1. union abc{
2.   int a;
3.   char b;
4.   float c;
5.   double d;
6. };
7. int main()
8. {
9.   printf("Size of union abc is %d", sizeof(union abc));
10.  return 0;
11. }
```

As we know, the size of int is 4 bytes, size of char is 1 byte, size of float is 4 bytes, and the size of double is 8 bytes. Since the double variable occupies the largest memory among all the four variables, so total 8 bytes will be allocated in the memory. Therefore, the output of the above program would be 8 bytes.

Accessing members of union using pointers

We can access the members of the union through pointers by using the (->) arrow operator.

Let's understand through an example.

```
1. #include <stdio.h>
2. union abc
3. {
4.   int a;
5.   char b;
6. };
7. int main()
8. {
9.   union abc *ptr; // pointer variable declaration
10.  union abc var;
11.  var.a= 90;
12.  ptr = &var;
```

```
13. printf("The value of a is : %d", ptr->a);
14. return 0;
15. }
```

In the above code, we have created a pointer variable, i.e., *ptr, that stores the address of var variable. Now, ptr can access the variable 'a' by using the (->) operator. Hence the output of the above code would be 90.

Why do we need C unions?

Consider one example to understand the need for C unions. Let's consider a store that has two items:

- Books
- Shirts

Store owners want to store the records of the above-mentioned two items along with the relevant information. For example, Books include Title, Author, no of pages, price, and Shirts include Color, design, size, and price. The 'price' property is common in both items. The Store owner wants to store the properties, then how he/she will store the records.

Initially, they decided to store the records in a structure as shown below:

```
1. struct store
2. {
3.     double price;
4.     char *title;
5.     char *author;
6.     int number_pages;
7.     int color;
8.     int size;
9.     char *design;
10. };
```

The above structure consists of all the items that store owner wants to store. The above structure is completely usable but the price is common property in both the items and the rest of the items are individual. The properties like price, *title, *author, and number_pages belong to Books while color, size, *design belongs to Shirt

REFERENCES:-

1. **Brian W.Kernighan and Dennis Ritchie,C Programming Language,Prantice Hall of India.**
2. **Yashwant Kanetkar,Let US C,BPB Publication.**